

A Semantic Query Optimiser using Automatic Rule Derivation

B G T Lowden, J Robinson and K Y Lim

*Department of Computer Science, The University of Essex,
Wivenhoe Park, Colchester CO4 3SQ, Essex,
United Kingdom
(lowdb@essex.ac.uk)*

Abstract: Semantic query optimization uses semantic knowledge to transform a query into another form that can be executed in a more efficient manner but still yields the same result as the original query. The semantic knowledge can be supplied by users or derived by the system.. In this paper, we describe the ARDOR semantic query optimizer with automatic rule derivation capabilities which, in recent field trials, has demonstrated significant reductions in query execution time.

1. Introduction

Automatic rule derivation for query reformulation has been proposed in previous work [3, 10,12,14]. In [7,15,16] the learning approach is driven mainly by a fixed set of heuristics based on the structure and implementation of the database. In contrast, the approach discussed in [9,13,17] places the emphasis on user queries, and data retrieved, to perform the rule identification needed for query reformulation. It is therefore likely to derive more useful rules since it is based on actual user requirements, which means the learning is closely related to the usage pattern of the database. Our approach extends this work in that we present a modified rule derivation mechanism based on example queries which minimizes learning time. Moreover, this approach requires no supervision and no intervention from users.

2. Dynamic Rules

Two classes of formulas are defined [1] which make up dynamic rules. Range information, which are *propositions* that assert the ranges of the values of database attributes e.g. $dcode \in ('ACCT', 'MKTG', 'PRSN')$, and the rules themselves which have an arbitrary number of range propositions on the antecedent side and one range proposition on the consequent side.

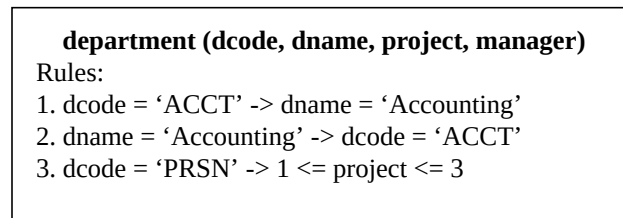


FIGURE 1. Rules Structure

In our approach, the semantic knowledge needed to perform the query reformulation consists of *simple* rules and the cost model of the database relations. *Simple* rules have only one range proposition on the antecedent portion and one range proposition on the consequent portion.

The purpose of this structure (Figure 1) is to enable rules to be matched more easily during the reformulation. It also minimises reformulation time since rules will only be fired if their

antecedent matches with the condition. Having only one range proposition in the consequent portion simplifies the learning process. Rules with set instances in either the antecedent or the consequent will not normally be derived since constraints created would be expensive to evaluate due to the high comparison cost involved in matching each term of the set. Examples of such rules are:

```
project > 12 -> dcode in ('MKTG', 'SALE')
dcode in ('MKTG', 'SALE') -> project > 12
```

3. Query Reformulation

Our query reformulation algorithm makes use of heuristic rules and reformulation operators as suggested in the previously cited work and summarised, below, with examples.

Constraint Addition

Introduce a constraint (preferably on an indexed attribute)

Original Query: Select * from student where entry = 94;

Rule: entry = 94 -> regno >= 940000

New Query: Select * from student where entry = 94 and regno >= 940000;

Constraint Deletion:

Removal of an unnecessary constraint.

Original Query: Select * from student where advisor = 100 and status = 'A';

Rule: status = 'A' -> advisor = 100

New Query: Select * from student where status = 'A';

Query Refutation:

Refute a query if it contradicts a rule.

Original Query: Select * from student where advisor = 105 and status = 'A';

Rule: status 'A' -> advisor = 100

New Query: Select null;

Figure 2 shows a simplified version of the algorithm. We illustrate the algorithm's operation by using the example relation "department" and its associated rules as given earlier in figure 1.

Consider the following query for reformulation:

```
select * from department where dname = 'Accounting' and manager = 'A01'
```

After the initial iteration, the query becomes:

```
select * from department where dname = 'Accounting' and manager = 'A01' and
dcode = 'ACCT'
```

Rules 1 and 2 of the three rules are fired in this reformulation process. At step 6, the highest cost condition is the condition *dname* = 'Accounting' because the attribute *dname* is not indexed and

Iterate steps 1 through 4 on all conditions C_i of the query, including new conditions added, until all relevant rules are fired.

1. Match condition C_i with antecedent A of each rule $A \rightarrow B$.
2. On matching, check the consequent B against the other conditions to see if the query is refuted.
If refuted terminate reformulation else proceed to 3.
3. Add the consequent to make new query condition C_n .
4. Create a dependency (C_n, C_i) where the new condition C_n is dependent on the current condition C_i which matches the rule.
5. Retrieve attribute cost information (eg. type, length and whether indexed) for each condition.
6. Repeat for all conditions: find the highest cost condition C_{high} from the list of conditions.
If there exists a dependency (C_{high}, C_i) then eliminate C_{high} and replace all dependencies (C_x, C_{high}) with (C_x, C_i) .

FIGURE 2. Query Reformulation Algorithm

has the longest length as compared to the other three attributes. Due to rule 1 being fired the dependency (dname, dcode) now holds and the latter condition is made redundant. The next highest cost condition is the condition $dcode = 'ACCT'$ which is retained since it does not form part of any dependency. Similarly, the next highest cost condition $manager = 'A01'$ is retained. Thus the reformulated query is:

*select * from department where manager = 'A01' and dcode = 'ACCT'*

This reformulation approach may easily be extended to queries which involve joining relations. In this case, the query is broken down into subqueries where each subquery performs a query on only one relation. Each subquery is reformulated independently of the other subqueries and the reformulated subqueries are then combined into the original query.

Although the tuples of each relation involved during the join are unchanged, time saved is substantial as the retrieval time of tuples from each relation is reduced. Moreover, in the event of any subquery being refuted, the reformulation process can automatically return a NULL answer since the refuted query will have no tuples satisfying the join condition.

4. Rule Derivation And Learning

In this section, we present our rule derivation and learning approach. We employ example user queries which trigger the system to derive semantic rules from the data in the database and learn rules that conform to the rules' structure as mentioned in section 2.

During the rule derivation phase, an alternative query is generated from the database using an inductive concept formation algorithm [2, 8]. This alternative query is semantically equivalent to the original query. In the rule learning phase, a series of operations are performed on the basic rules to transform them into the required structure. A check is made for duplicates and rules similar to those already stored are identified.

We first present the algorithm for learning the alternative query. The instances in the target relation are divided into positive examples, which are the answers returned by the query, and the remaining

instances are the negative examples [5, 6]. The alternative query formed should be satisfied only by the answer instances and none of the others. This is necessary to guarantee that the original query and the alternative query are equivalent. Figure 3 shows the algorithm to learn the alternative query, which would normally run as a background task.

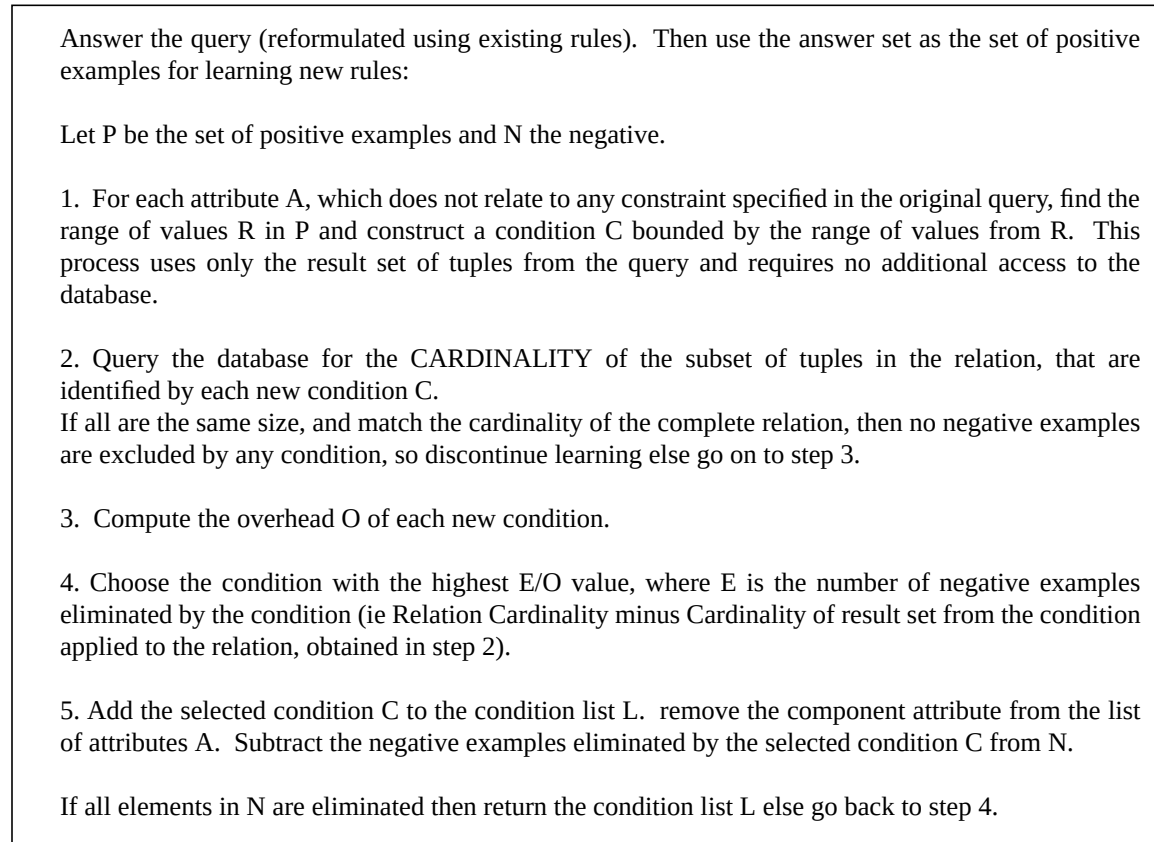


FIGURE 3. Inductive Learning Algorithm For The Alternative Query

The learning algorithm does not construct new conditions based on attributes specified in the conditions of the original query. This avoids the derivation of trivial rules with similar attributes in both the antecedent and consequent and ensures that the time spent on deriving useful rules is minimised.

In the subsequent rule derivation phase, described later, rules are derived by setting the conditions of the original query equal to that of the alternative query.

To illustrate the way in which the algorithm operates we use an example database relation *department* shown in Table 1. This relation has 15 instances and the 4 attributes are: *dcode*(char4), *dname*(char15), *project*(integer), *manager*(char6). The *project* attribute is indexed.

The query for the relation is:

*select * from department where dname = 'Marketing'*

Shaded tuples indicate positive examples of the above query (i.e. the result set for the query).

dcode	dname	project	manager
ACCT	Accounting	1	ACCT01
ACCT	Accounting	1	ACCT02
ACCT	Accounting	5	ACCT01
MKTG	Marketing	2	MKTG01
MKTG	Marketing	2	MKTG04
MKTG	Marketing	2	MKTG05
MKTG	Marketing	3	MKTG02
MKTG	Marketing	4	MKTG03
MKTG	Marketing	5	MKTG01
MKTG	Marketing	5	MKTG04
MKTG	Marketing	5	MKTG05
MKTG	Marketing	6	MKTG02
MKTG	Marketing	6	MKTG03
PRSN	Personnel	7	PRSN01
PRSN	Personnel	8	PRSN02

TABLE 1. Example Database Relation DEPARTMENT

In step 1, the system extracts range values for each attribute from the positive instances and represents them in a condition. In step 2, the number of negative examples eliminated by each condition is counted and the overheads of each condition computed. Table 2a shows the conditions and the computation.

Conditions	E	CO	TCO	O = CO * TCO	E/O
dcode = 'MKTG'	5	4	15	60	0.0833
project ≥ 2 and project ≤ 6	4	4	11	40	0.1000
manager in ('MKTG01', MKTG02',)	5	30	15	450	0.0111

E denotes number of negative examples eliminated. **CO** denotes comparison overhead.
TCO denotes tuple count overhead. **O** denotes overhead.

TABLE 2a. Conditions and Computation

The overhead of a condition is the comparison overhead CO multiplied by the number of tuples to

be evaluated TCO. The CO is the overhead to evaluate whether a tuple satisfies the condition. This overhead is the number of terms to be compared in the given condition multiplied by the overhead for each comparison. For strings, the comparison overhead of each term is their length as defined in the relation. For integers, the comparison overhead depends on the platform the query optimizer is running on. In our case, it is 2. For conditions on indexed attributes, TCO is the number of tuples that satisfy the condition, else it is the total number of tuples in the relation, since every tuple in the relation must be scanned. Thus, the condition on *project* has a CO of 4 since there are two terms to compare and each is of integer type with comparison overhead 2. The TCO of *project* is 10 because it is indexed. Thus, the overhead for *project* is $4 \times 10 = 40$. The CO of the condition on *manager* is 30 since there are five terms to compare and each has a comparison overhead of 6.

At step 4, the condition on the attribute *project* is selected because it has the highest E/O ratio. At step 5, this condition is added to the condition list and the component attributes removed from the list of attributes eligible for computation of the E and O. The number of negative examples is now only 1 because the selected condition on *project* eliminates 4 of the 5 negative examples. The whole process from step 2 is iterated and Table 2b shows the updated computation for the remaining 3 constraints.

The condition on *dcode* is then selected thus eliminating the remaining negative example and the learning of the alternative query is completed. The alternative query is therefore:

*select * from department where project ≥ 2 and project ≤ 6 and dcode = 'MKTG'*

Conditions	E	CO	TCO	O = CO * TCO	E/O
dcode = 'MKTG'	1	4	11	44	0.0227
manager in ('MKTG01', 'MKTG02',)	1	30	11	330	0.0030

TABLE 2b. Updated Conditions and Computation

Following induction of the alternative query, the rule learning phase performs the transformation and learning of the rules. This process takes the derived rules from both the original query and alternative query, transforms and learns them.

First, the process uses a logical inference procedure to transform the original/alternative query equivalence into dynamic rules formulated with a single range proposition on both the antecedent and consequent side. Using the original query and the alternative query in the earlier example, the equivalence in the two conditions gives:

$$\text{dname} = \text{'Marketing'} \Leftrightarrow \text{project} \geq 2 \wedge \text{project} \leq 6 \wedge \text{dcode} = \text{'MKTG'}$$

The equivalence of the two conditions is converted into two implication rules:

- (1) $\text{dname} = \text{'Marketing'} \Rightarrow \text{project} \geq 2 \wedge \text{project} \leq 6 \wedge \text{dcode} = \text{'MKTG'}$
- (2) $\text{project} \geq 2 \wedge \text{project} \leq 6 \wedge \text{dcode} = \text{'MKTG'} \Rightarrow \text{dname} = \text{'Marketing'}$

Rule (1) is further expanded to

- (1a) $\text{dname} = \text{'Marketing'} \Rightarrow \text{project} \geq 2$
- (1b) $\text{dname} = \text{'Marketing'} \Rightarrow \text{project} \leq 6$
- (1c) $\text{dname} = \text{'Marketing'} \Rightarrow \text{dcode} = \text{'MKTG'}$

If the rule has more than one antecedent, e.g. rule (2), the system will use the greedy minimum set cover algorithm [4] to eliminate unnecessary constraints. Rule (2) can be represented in an equivalent way:

$$\neg(\text{project} \geq 2) \vee \neg(\text{project} \leq 6) \vee \neg(\text{dcode} = \text{'MKTG'}) \Rightarrow \neg(\text{dname} = \text{'Marketing'})$$

The minimum number of sets that cover the set of instances that satisfy $\neg(\text{dname} = \text{'Marketing'})$ is required and this must be 1 in order to obey the syntax requirement of having one range proposition on the antecedent side. In this example, the minimum set that covers $\neg(\text{dname} = \text{'Marketing'})$ is $\neg(\text{dcode} = \text{'MKTG'})$, the range propositions $\neg(\text{project} \geq 2)$ and $\neg(\text{project} \leq 6)$ can be eliminated and the rule formed is:

$$(2a) \text{dcode} = \text{'MKTG'} \Rightarrow \text{dname} = \text{'Marketing'}$$

Therefore, rules (1a), (1b), (1c) and (2a) are returned.

Each new derived rule is first checked against those that are in the the current rule set to ensure that it does not already exist. If so, then the derived rule is ignored. If the new rule does not exist, then a refinement is done on similar rules where the existing rules and the new rules differ only in the operators or constant values in either the antecedent or consequent side.

The antecedent side of the rule keeps its constant value boundary as wide as possible whereas the consequent side of the rule keeps its constant value boundary as small as possible. The aim is to have as wide a boundary antecedent as possible to enable the rule to be matched more easily and a small boundary consequent to ensure that any new constraint added to the query has a small range. The latter also means that new conditions, created by the consequent side of the rule, will find antecedents of other rules easier to match because of its narrow range.

Consider two existing rules:

- (1) $\text{project} \geq 12 \Rightarrow \text{dcode} = \text{'SALE'}$
- (2) $\text{dcode} = \text{'SALE'} \Rightarrow \text{project} \geq 12$

Assume two rules similar to rule (1) are derived and are to be stored in the file:

- (1a) $\text{project} > 15 \Rightarrow \text{dcode} = \text{'SALE'}$
- (1b) $\text{project} \geq 10 \Rightarrow \text{dcode} = \text{'SALE'}$

Rule (1a) is ignored because, by changing the antecedent of rule(1) to that of rule(1a), the boundary of the rule becomes smaller ie. from (≥ 12) to (> 15). The antecedent of rule (1b) is used to replace that of rule (1) since using the new antecedent means that the boundary of the antecedent is expanded. Thus rule (1b) replaces rule (1).

Assume, now rules similar to rule (2) are derived and are to be stored in the file:

(2a) dcode = 'SALE' $\Rightarrow$ project > 15

(2b) dcode = 'SALE' $\Rightarrow$ project $\geq$ 10

In this case, rule (2b) is ignored and rule (2a) is chosen to replace rule (2) since using the new consequent means that the boundary is reduced to a smaller range, ie. from (≥ 12) to (≥ 15).

5. Conclusion

Currently, our ARDOR system is still in the experimental stage though initial field trials show an average saving of about 35% (compared with the same queries executed without S.Q.O.) over several hundred reformulated queries, on the University student database of around 3×10^4 tuples.

The method requires no initialisation overhead. It generates its first rules after the first query. The learning method is query-based so that learned rules are relevant to query optimisation. The restriction to dependencies between attributes in the same relation limits workload and so improves efficiency, but applies directly to queries with multiple relations.

We recognise that an important aspect of future work is to reduce the learning time for large databases and are currently looking into the use of statistical techniques to identify the correlation between attribute pairs to eliminate weak dependencies.

This approach could also be extended to assign coefficients of relevance to each rule class which would assist in maintaining a limit on the size of the active rule set. Another aspect of rule maintenance is to ensure that rules are consistent with the state of a changed database. We have proposed algorithms [11] which ensure that affected rules are not deleted unless they become invalid with respect to the new state.

The method does not require data retrieval apart from the query result set to learn rules involving only two attributes (i.e. rewrite rules to replace single conditions in a query with less costly single conditions). The only database access in this case is a cardinality query, which avoids the data retrieval overhead (significant in distributed and multiprocessor servers) as well as the subsequent data-intensive task of comparing sets of values returned by different queries.

References

- [1] Arens Y. et al. (1994) Query Processing in an Information Mediator. *Proceedings of the ARPA Planning Initiative Meeting*.
- [2] Cai Y., Cerone N. and Han J. (1991) Learning in relational databases: An attribute-oriented approach. *Computational Intelligence J.*, **7(3)**, 119-132.
- [3] Chakravarthy U., Grant J. and Minker J. (1990) Logic based approach to semantic query optimisation. *ACM Transactions on Database Systems*, **15(2)**, 162-207.
- [4] Cormen T., Leiserson C. and Rivest R. (1989) Introduction to algorithms. *MIT Press/McGraw-Hill Book Co.*, Cambridge, MA.

- [5] Dietterich T. and Michalski R. (1983) A comparative review of selection methods for learning from examples. *Machine Learning: An Artificial Intelligence Approach*, (1), 41-82.
- [6] Geneesereth M. and Nilsson N. (1987) *Logical Foundations of Artificial Intelligence*. Morgan Kaufmann.
- [7] Han J., Cai Y. and Cercone N. (1993) Data driven discovery of quantitative rules in relational databases. *IEEE Trans. on Knowledge and Data Engineering*, 5(1), 29-40.
- [8] Haussler D. (1988) Quantifying inductive bias: AI learning algorithms and Valiant's learning framework. *Artificial Intelligence J.*, **36**, 177-221.
- [9] Hsu C. and Knoblock C. (1993) Learning database abstractions for query reformulation. *Proceedings of the AAAI Workshop on Knowledge Discovery in Databases*, 276-290.
- [10] King J. (1981) QUIST: A system for semantic query optimisation in relational databases. *Proc. 7th VLDB Conf.*
- [11] Lowden B. and Lim K. (1994) ARDOR: A query optimiser using automatic rule derivation. *Computer Science Technical Report CSM 211*, University of Essex, December 1994.
- [12] Pang H., Lu H. and Ooi B. (1991) An efficient semantic query optimisation algorithm. *Proc. Data Engineering Conf. IEEE*.
- [13] Shekhar S., Hamidzadeh B. and Kohli A. (1993) Learning transformation rules for semantic query optimisation: a data driven approach. *IEEE Trans. Knowledge and Data Eng.*, 5(6), 949-964.
- [14] Shenoy S. and Ozsoyoglu Z. (1989) Design and implementation of a semantic query optimizer. *IEEE Transactions on Knowledge and Data Engineering*, **1(3)**, 344-361.
- [15] Siegel M. (1988) Automatic rule derivation for semantic query optimisation. *Proc. 2nd. Int. Conf. on Expert Database Systems*. 371-385.
- [16] Siegel M., Sciore E., and Salveter S. (1992) A method for automatic rule derivation to support semantic query optimization. *ACM Trans. on Database Systems*, **17(4)**, 563-600.
- [17] Yu C. and Sun W. (1989) Automatic knowledge acquisition and maintenance for semantic query optimisation. *IEEE Trans. Knowledge and Data Processing*, 362-375.

